Bài 2.10 – Random Forest (Rừng ngẫu nhiên)

> **Giai đoạn 2 – Machine Learning cổ điển** · Thời lượng: 1 buổi (2–3 giờ) · Hình thức mở đầu: *Câu chuyện*

🎯 Sau bài này bạn sẽ

- Hiểu vì sao **nhiều mô hình yếu** gộp lại thành **một mô hình mạnh** (sức mạnh đám đông).
- Hiểu hai nguồn "ngẫu nhiên" của Random Forest: **bootstrap** và **chọn ngẫu nhiên đặc trưng**.
- Dùng **OOB score** để tự đánh giá mà không cần tập validation.
- Phân biệt hai loại feature importance, và biết loại nào đáng tin hơn.
- Đạt kết quả tốt nhất từ đầu khóa đến giờ cho bài toán giá nhà.

---

1. Câu chuyện: Đoán số hạt đậu trong lọ

Năm 1906, nhà khoa học Francis Galton đến một hội chợ gia súc ở Anh. Có cuộc thi đoán cân nặng một con bò. Khoảng 800 người tham gia: nông dân, đồ tể, cả người chưa từng nuôi bò.

Từng người đoán **sai khá nhiều**. Nhưng khi Galton lấy **giá trị giữa** (trung vị) của tất cả các dự đoán, kết quả chỉ lệch khoảng **1%** so với cân nặng thật, tốt hơn cả các chuyên gia.

Bí quyết: mỗi người sai **theo một kiểu khác nhau**. Người đoán cao, người đoán thấp. Gộp lại, các sai số **triệt tiêu nhau**. Người ta gọi đó là **"trí tuệ đám đông"**.

Nhưng có một điều kiện: đám đông phải **đa dạng và độc lập**. Nếu 800 người cùng nghe một "chuyên gia" rồi đoán theo, họ sẽ sai **giống hệt nhau** và không triệt tiêu được gì.

Random Forest áp dụng đúng ý tưởng này: trồng **hàng trăm cây quyết định khác nhau**, rồi cho chúng bỏ phiếu.

---

2. Vấn đề của một cây đơn lẻ

Bài 2.9 đã cho thấy: cây quyết định **không ổn định** (variance cao). Lấy 3 mẫu bootstrap khác nhau (mục 3.1) từ cùng dữ liệu tín dụng, huấn luyện 3 cây độ sâu 3:

```
import numpy as np
import pandas as pd
from sklearn.datasets import fetch_openml
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import roc_auc_score

credit = fetch_openml("credit-g", version=1, as_frame=True).frame
credit["vo_no"] = (credit["class"] == "bad").astype(int)
X = pd.get_dummies(credit.drop(columns=["class", "vo_no"]), drop_first=True,
dtype=int)
y = credit["vo_no"]
```

```
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.2, random_state=42,
stratify=y)

for seed in [1, 2, 3]:
    Xb = Xtr.sample(frac=1, replace=True, random_state=seed)    # bốc ngẫu nhiên
CÓ HOÀN LẠI
    yb = ytr.loc[Xb.index]                                      # lấy nhãn tương
ứng
    t = DecisionTreeClassifier(max_depth=3, random_state=0).fit(Xb, yb)
    print(seed, X.columns[t.tree_.feature[0]],                  # câu hỏi ở gốc
          round(roc_auc_score(yte, t.predict_proba(Xte)[:, 1]), 3))
```

Kết quả mẫu:

```
1 checking_status_no checking 0.726
2 checking_status_no checking 0.692
3 checking_status_no checking 0.666
```

Cùng câu hỏi gốc, nhưng AUC dao động từ 0,666 đến 0,726 chỉ vì dữ liệu huấn luyện xáo trộn khác đi một chút. Mỗi cây là một "người đoán" **thiếu ổn định**. Gộp nhiều cây lại sẽ ổn định hơn.

---

3. Hai nguồn đa dạng của Random Forest

3.1. Bootstrap (Lấy mẫu có hoàn lại)

- **Định nghĩa:** Từ n mẫu, bốc ngẫu nhiên n lần, **mỗi lần bốc xong bỏ lại vào**. Một mẫu có thể bị bốc nhiều lần, mẫu khác không được bốc lần nào.
- **Ví dụ đời thực:** Bốc thăm 5 lần từ hộp 5 lá thăm A–E, mỗi lần bốc xong bỏ lại vào hộp.

```
rng = np.random.default_rng(0)
hop = np.array(["A", "B", "C", "D", "E"])
for i in range(3):
    print(rng.choice(hop, size=5, replace=True))     # replace=True: có hoàn lại
```

Kết quả mẫu:

```
['E' 'D' 'C' 'B' 'B']
['A' 'A' 'A' 'A' 'E']
['D' 'E' 'C' 'D' 'E']
```

Mỗi "bộ dữ liệu bootstrap" khác nhau → mỗi cây học từ một góc nhìn khác nhau.

Bao nhiêu % mẫu gốc được chọn?

```
n = 1000
ti_le = [len(np.unique(rng.integers(0, n, n))) / n for _ in range(200)]   # 200
lần thử
print(round(np.mean(ti_le), 3), round(1 - (1 - 1 / n) ** n, 3))           # mô
phỏng vs công thức
```

Kết quả mẫu:

```
0.633 0.632
```

Mỗi cây chỉ thấy khoảng **63%** số mẫu khác nhau. **37% còn lại** gọi là mẫu **out-of-bag (OOB)**: cây đó chưa từng thấy chúng. Ta sẽ tận dụng điều này ở mục 4.3.

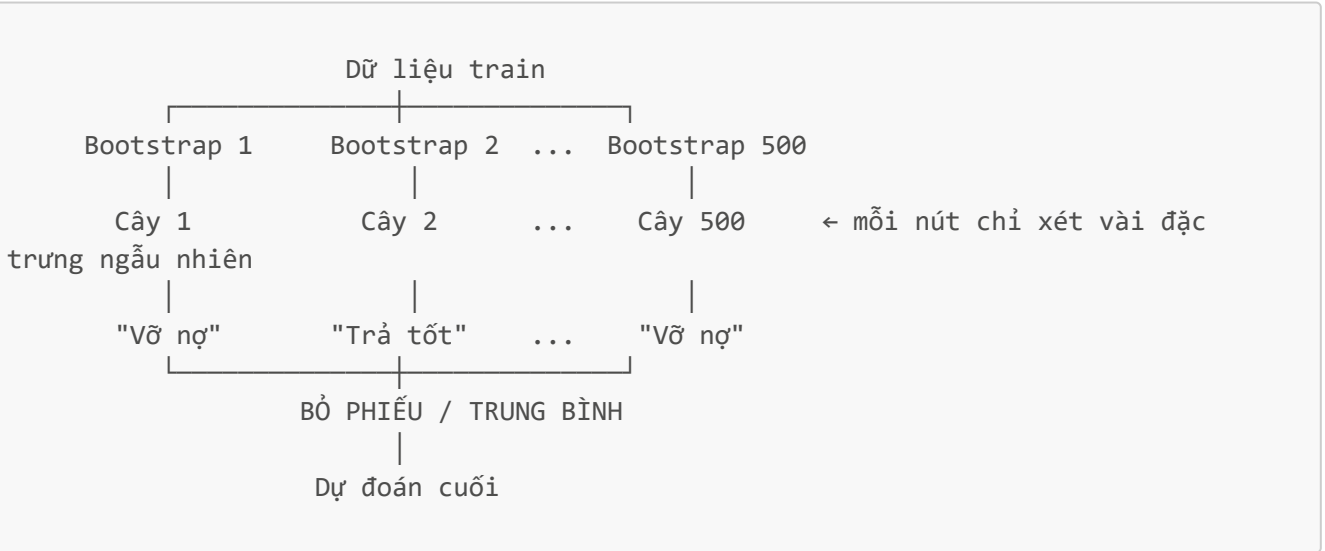
(Giải thích công thức: xác suất một mẫu **không** được chọn trong 1 lần bốc là (1 − 1/n). Không được chọn trong cả n lần: $(1 - 1/n)^n \approx e^{-1} \approx 0{,}368$.)

Bagging = Bootstrap + Aggregating (gộp): huấn luyện nhiều mô hình trên các bộ bootstrap khác nhau, rồi gộp kết quả (bỏ phiếu cho phân loại, lấy trung bình cho hồi quy).

3.2. Chọn ngẫu nhiên đặc trưng ở mỗi nút

Chỉ bootstrap thôi chưa đủ. Nếu có một đặc trưng **rất mạnh** (như `checking_status`), **mọi cây** đều chọn nó ở gốc → các cây vẫn giống nhau → sai giống nhau.

Random Forest thêm một bước: ở **mỗi nút**, cây chỉ được xét một **tập con ngẫu nhiên** các đặc trưng (mặc định √(số đặc trưng) với phân loại). Với 48 đặc trưng, mỗi nút chỉ được xét khoảng 7 cột ngẫu nhiên. Đôi khi đặc trưng mạnh nhất không có mặt, cây buộc phải tìm câu hỏi khác → các cây **thật sự đa dạng**.



---

4. Random Forest cho bài toán tín dụng

4.1. Huấn luyện

```
from sklearn.ensemble import RandomForestClassifier

rf = RandomForestClassifier(n_estimators=500,      # số cây
                            n_jobs=-1,             # dùng mọi nhân CPU để chạy
song song
                            oob_score=True,        # tính điểm trên mẫu out-of-bag
                            random_state=42).fit(Xtr, ytr)
print("Acc train:", round(rf.score(Xtr, ytr), 3), "| Acc test:",
round(rf.score(Xte, yte), 3),
      "| AUC test:", round(roc_auc_score(yte, rf.predict_proba(Xte)[:, 1]), 3),
      "| OOB:", round(rf.oob_score_, 3))
```

Kết quả mẫu:

```
Acc train: 1.0 | Acc test: 0.77 | AUC test: 0.799 | OOB: 0.764
```

So sánh:

Mô hình	AUC test
Một cây tốt nhất (bài 2.9)	0,711
Random Forest 500 cây	**0,799**
Logistic Regression 6 cột (bài 2.5)	0,803

- Rừng vượt xa một cây.
- Logistic Regression vẫn ngang ngửa: với dữ liệu nhỏ (800 mẫu train) và quan hệ khá "thẳng", mô hình đơn giản vẫn rất cạnh tranh. **Đừng mặc định mô hình phức tạp sẽ thắng.**

4.2. Train 100% mà vẫn ổn?

Mỗi cây trong rừng mọc **không giới hạn độ sâu** nên học thuộc lòng dữ liệu của nó, và cả rừng đạt 100% trên train. Nhưng khác với một cây đơn lẻ (test 62%), rừng vẫn đạt 77% test. Vì sao? Mỗi cây học vẹt **theo một kiểu khác nhau**; khi gộp lại, các "kiểu học vẹt" **triệt tiêu nhau**, chỉ còn quy luật chung được giữ lại. Đây là cách Random Forest **giảm variance** mà không tăng bias.

> ⚠️ Vì vậy, với Random Forest, **accuracy train không cho biết gì** về overfitting. Hãy nhìn OOB hoặc validation.

4.3. OOB score: "tập validation miễn phí"

Với mỗi khách trong train, khoảng 37% số cây **chưa từng thấy** khách đó (mục 3.1). Cho những cây đó bỏ phiếu → được một dự đoán "như trên dữ liệu mới". Làm vậy cho mọi khách → **OOB score**.

OOB = 0,764, rất gần accuracy test 0,77. Khi dữ liệu ít, OOB giúp bạn **không phải cắt riêng** một phần train để làm validation.

4.4. Bao nhiêu cây là đủ?

```
for n_cay in [1, 10, 50, 100, 300, 1000]:
    m = RandomForestClassifier(n_estimators=n_cay, random_state=42,
n_jobs=-1).fit(Xtr, ytr)
    print(f"{n_cay:>4} cây: AUC test = {roc_auc_score(yte, m.predict_proba(Xte)[:,
1]):.3f}")
```

Kết quả mẫu:

```
   1 cây: AUC test = 0.589
  10 cây: AUC test = 0.719
  50 cây: AUC test = 0.772
 100 cây: AUC test = 0.773
 300 cây: AUC test = 0.797
1000 cây: AUC test = 0.798
```

- Thêm cây **không làm overfit**: kết quả tăng rồi đi ngang.
- Từ 300 lên 1.000 cây gần như không thêm gì, chỉ tốn thời gian gấp 3.
- **Mẹo:** 200–500 cây thường là đủ.

4.5. Các tham số quan trọng

Tham số	Ý nghĩa	Gợi ý
`n_estimators`	Số cây	200–500
`max_features`	Số đặc trưng xét ở mỗi nút	`"sqrt"` (mặc định phân loại); nhỏ hơn → cây đa dạng hơn
`max_depth`, `min_samples_leaf`	Giới hạn mỗi cây	Mặc định không giới hạn; tăng `min_samples_leaf` khi dữ liệu nhiễu
`class_weight`	Trọng số cho lớp hiếm	`"balanced"` khi lớp dương rất ít
`n_jobs`	Số nhân CPU	`-1` = dùng hết

5. Feature importance: hai cách, hai câu chuyện

5.1. Cách 1: dựa trên Gini (mặc định, như bài 2.9)

```
imp = pd.Series(rf.feature_importances_,
index=X.columns).sort_values(ascending=False)
print(imp.head(8).round(3))
```

Kết quả mẫu:

```
credit_amount                  0.114
age                            0.087
duration                       0.087
checking_status_no checking    0.059
installment_commitment         0.042
residence_since                0.038
checking_status_<0             0.036
savings_status_<100            0.025
dtype: float64
```

Số tiền vay và tuổi đứng đầu? Lạ, vì bài 2.9 cho thấy `checking_status` mạnh nhất. Nguyên nhân: các cột số có **rất nhiều giá trị khác nhau** (số tiền vay có hàng trăm giá trị) nên cây có **nhiều ngưỡng để thử**, dễ tìm được ngưỡng "tình cờ" làm giảm Gini trên dữ liệu train (kể cả do nhiễu). Cách đo này **thiên vị cột nhiều giá trị**.

5.2. Cách 2: Permutation importance (Độ quan trọng hoán vị)

- **Định nghĩa:** Xáo trộn ngẫu nhiên **một cột** trên tập test (phá vỡ mối liên hệ của nó với y), rồi xem điểm của mô hình **giảm bao nhiêu**. Giảm nhiều → cột đó quan trọng.
- **Ví dụ đời thực:** Muốn biết cầu thủ nào quan trọng nhất, cho từng người ngồi dự bị một trận rồi xem đội thua bao nhiêu bàn.

```
from sklearn.inspection import permutation_importance

pi = permutation_importance(rf, Xte, yte, scoring="roc_auc", n_repeats=10,
random_state=42, n_jobs=-1)
pis = pd.Series(pi.importances_mean, index=X.columns).sort_values(ascending=False)
print(pis.head(8).round(3))
print(pis.tail(3).round(3))
```

Kết quả mẫu:

```
checking_status_no checking                     0.069
duration                                        0.018
checking_status_<0                              0.018
credit_amount                                   0.017
purpose_new car                                 0.013
credit_history_critical/other existing credit   0.012
age                                             0.012
other_payment_plans_none                        0.009
dtype: float64
purpose_radio/tv    -0.003
employment_<1       -0.004
```

```
purpose_education   -0.006
dtype: float64
```

- Xáo trộn `checking_status_no checking` → AUC giảm **0,069**, gấp gần 4 lần cột đứng thứ hai. Khớp với bài 2.4 và 2.9.
- `age` tụt từ hạng 2 xuống hạng 7.
- Vài cột có giá trị **âm**: xáo trộn chúng thì mô hình còn **tốt hơn một chút**. Tức là chúng gần như vô dụng (hoặc gây nhiễu).

	Gini importance	**Permutation importance**
Tính trên	Train (lúc học)	**Test / validation**
Thiên vị cột nhiều giá trị	**Có**	Không
Tốc độ	Tức thì	Chậm hơn
Đáng tin	Vừa phải	**Hơn**

> ⚠️ Cả hai cách đều có điểm yếu khi hai đặc trưng **tương quan mạnh**: xáo trộn một cột, cột kia vẫn "gánh" thông tin, nên cả hai đều trông kém quan trọng.

---

6. Random Forest cho giá nhà

```
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error

df = nap_du_lieu_nha()     # ô chuẩn bị dữ liệu ở bài 2.1
cot = ["thu_nhap", "km_tp_lon", "phong_moi_ho", "ti_le_phong_ngu",
       "nguoi_moi_ho", "tuoi_nha", "kinh_do", "vi_do"]
Xtr, Xte, ytr, yte = train_test_split(df[cot], df["gia_nha"], test_size=0.2,
random_state=42)
rfr = RandomForestRegressor(n_estimators=300, n_jobs=-1, oob_score=True,
random_state=42).fit(Xtr, ytr)
print("R² train:", round(rfr.score(Xtr, ytr), 3), "| R² test:",
round(rfr.score(Xte, yte), 3),
      "| MAE test:", round(mean_absolute_error(yte, rfr.predict(Xte)), 1), "| OOB
R²:", round(rfr.oob_score_, 3))
print(pd.Series(rfr.feature_importances_,
index=cot).sort_values(ascending=False).round(3))
```

Kết quả mẫu (khoảng vài giây):

```
R² train: 0.973 | R² test: 0.793 | MAE test: 29.1 | OOB R²: 0.797
thu_nhap           0.425
km_tp_lon          0.166
nguoi_moi_ho       0.138
kinh_do            0.082
```

```
vi_do              0.079
tuoi_nha           0.041
ti_le_phong_ngu    0.034
phong_moi_ho       0.033
dtype: float64
```

Bảng thành tích giá nhà từ đầu Giai đoạn 2:

Mô hình	R^2 test	MAE test (nghìn USD)
Mốc (đoán trung bình)	0,000	77,6
Linear Regression 6 cột	0,537	48,7
Ridge đa thức bậc 3	0,672	
Một cây (độ sâu 10)	0,665	37,9
KNN chỉ dùng vị trí		30,9
Random Forest	**0,793**	**29,1**

`km_tp_lon` (đặc trưng tự tạo ở bài 1.12) đứng thứ 2. Công sức EDA đã được đền đáp.

Rừng cũng cho biết "mức độ chắc chắn"

```
du_doan_tung_cay = np.array([c.predict(Xte.values[:1])[0] for c in
rfr.estimators_])  # 300 cây đoán khu đầu tiên
print(du_doan_tung_cay[:5].round(1), "| TB:", round(du_doan_tung_cay.mean(), 1),
      "| rừng:", round(rfr.predict(Xte[:1])[0], 1), "| thật:", round(yte.iloc[0],
1),
      "| ĐLC:", round(du_doan_tung_cay.std(), 1))
```

Kết quả mẫu:

```
[66.1 57.4 49.4 84.4 64.9] | TB: 55.6 | rừng: 55.6 | thật: 52.6 | ĐLC: 9.7
```

Từng cây đoán khá xa nhau (49 đến 84), nhưng trung bình 300 cây (55,6) rất gần giá thật (52,6). Độ lệch chuẩn giữa các cây (9,7) là một gợi ý về **mức độ không chắc chắn** của dự đoán: khu nào các cây "cãi nhau" nhiều thì nên cẩn thận hơn.

---

7. Ưu, nhược điểm

Ưu điểm	Nhược điểm
Rất mạnh "ngay khi mở hộp", ít cần chỉnh tham số	Mất tính dễ đọc của một cây (500 cây không ai đọc nổi)

Ưu điểm	Nhược điểm
Khó overfit khi thêm cây	Mô hình nặng (lưu hàng trăm cây), dự đoán chậm hơn mô hình tuyến tính
Không cần chuẩn hóa, chịu outlier tốt	Hồi quy: không dự đoán được ngoài khoảng giá trị đã thấy
Có OOB, có feature importance	Thường thua Gradient Boosting một chút (bài 2.11)
Chạy song song nhiều nhân CPU	

8. Bài tập

1. Một Random Forest có 3 cây dự đoán xác suất vỡ nợ của một khách: 0,6; 0,3; 0,8. Xác suất cuối cùng là bao nhiêu? Với ngưỡng 0,5, quyết định là gì?
2. Thử `RandomForestClassifier(max_features=0.5)` (mỗi nút xét 50% số đặc trưng) và `max_features=1` (chỉ 1 đặc trưng ngẫu nhiên). AUC test thay đổi thế nào? Giải thích theo khái niệm "đa dạng".
3. Thử `class_weight="balanced"` cho bài toán tín dụng. Recall của lớp vỡ nợ (ngưỡng 0,5) thay đổi thế nào?
4. Tính permutation importance cho mô hình giá nhà (dùng `scoring="neg_mean_absolute_error"`). Thứ tự có khác Gini importance không?

Đáp án

> Mẹo Notion: chọn tiêu đề "Đáp án" → `Turn into → Toggle heading 3`.

1. Trung bình: (0,6 + 0,3 + 0,8) / 3 ≈ **0,567** → ≥ 0,5 → **vỡ nợ**. (Scikit-learn gộp bằng cách lấy trung bình xác suất, không phải đếm phiếu cứng.)
2. Code:

```
for mf in [1, "sqrt", 0.5]:
    m = RandomForestClassifier(n_estimators=500, max_features=mf, random_state=42,
n_jobs=-1).fit(Xtr_c, ytr_c)
    print(mf, round(roc_auc_score(yte_c, m.predict_proba(Xte_c)[:, 1]), 3))
```

(Dùng lại X, y tín dụng ở mục 2, đặt tên `Xtr_c`... vì mục 6 đã ghi đè `Xtr`.) `max_features` nhỏ → cây đa dạng hơn nhưng mỗi cây yếu hơn. Lớn → cây mạnh hơn nhưng giống nhau hơn. Thường có một điểm cân bằng ở giữa; ghi lại kết quả thật để so sánh. 3. `class_weight="balanced"` làm mô hình "coi trọng" lớp vỡ nợ hơn (vì lớp này ít) → **recall lớp vỡ nợ thường tăng**, precision giảm. Tác dụng tương tự hạ ngưỡng ở bài 2.5. 4. Code:

```
pi_nha = permutation_importance(rfr, Xte, yte, scoring="neg_mean_absolute_error",
n_repeats=5, random_state=42, n_jobs=-1)
print(pd.Series(pi_nha.importances_mean,
index=cot).sort_values(ascending=False).round(2))
```

Đọc: mỗi số là MAE **tăng thêm** bao nhiêu nghìn USD khi xáo trộn cột đó. So thứ tự với Gini importance ở mục 6. Thường các cột vị trí (`kinh_do`, `vi_do`) lên hạng: Gini chia "công lao vị trí" cho 3 cột tương quan (`km_tp_lon`, `kinh_do`, `vi_do`), còn permutation đo riêng từng cột.

---

9. Flashcard

Mặt trước	**Mặt sau**
Ý tưởng của Random Forest?	Nhiều cây đa dạng bỏ phiếu → sai số triệt tiêu nhau.
Hai nguồn đa dạng?	Bootstrap mẫu + chọn ngẫu nhiên đặc trưng ở mỗi nút.
Bootstrap là gì?	Lấy n mẫu có hoàn lại từ n mẫu gốc.
Mỗi cây thấy khoảng bao nhiêu % mẫu?	~63%. 37% còn lại là out-of-bag.
OOB score là gì?	Điểm tính bằng các cây chưa thấy mẫu đó; như validation miễn phí.
Thêm cây có làm overfit?	Không; kết quả tăng rồi đi ngang.
Random Forest giảm bias hay variance?	Chủ yếu giảm variance.
Gini importance thiên vị gì?	Cột có nhiều giá trị khác nhau.
Permutation importance đo gì?	Điểm giảm bao nhiêu khi xáo trộn một cột trên test.
Random Forest có cần chuẩn hóa?	Không.

Bài tiếp theo: 2.11 – Gradient Boosting: XGBoost, LightGBM.